Quant Research and Trading Recommendation Agent

Product Requirements Document (PRD) and Technical Specification

Prepared for implementation planning

Version	v1.0
Date	10 April 2026
Document owner	Product / Engineering
Scope	MVP through production-grade research workflow

Purpose: define a quant-focused decision-support agent that turns market information and price behavior into explainable stock ideas, recommended entry zones, sell targets, and risk-aware invalidation rules.

1. Executive Summary

This document specifies a quant research and trading recommendation agent that monitors market data, news, filings, and price action to surface candidate equities, rank them, and propose trade plans. The product is designed as a decision-support system rather than an unmanaged auto-trading black box. It combines deterministic analytics, backtested signal logic, and controlled use of LLMs for interpretation and explanation.

The system is intended to answer four practical questions each trading day: which stocks are interesting now, why they are interesting, where they may be bought or sold, and under what conditions the thesis is invalidated.

2. Product Vision

Build a research-grade agent that can continuously convert fast-moving market information into structured, auditable, and risk-screened trade recommendations suitable for analyst review, paper trading, and eventual supervised execution.

3. Goals and Non-Goals

Goals	Non-Goals
Generate ranked stock ideas from blended market information and price features.	Do not promise profitable outcomes or replace fiduciary investment advice.
Produce actionable price zones: entry, stop, target, and expected holding period.	Do not let the LLM invent price levels without deterministic logic.
Maintain a full audit trail with input snapshots, model outputs, and human approvals.	Do not submit live orders in the MVP without explicit user approval.
Support paper trading, walk-forward validation, and post-trade attribution.	Do not optimize solely for backtest returns at the expense of realism.

4. Primary Users

- Independent trader or analyst who wants a short list of high-quality ideas instead of scanning the entire market manually.
- Quant product owner who needs a modular architecture that can evolve from research to production.
- Engineering team responsible for data pipelines, agent orchestration, and monitoring.
- Risk reviewer who needs traceability, policy controls, and measurable performance.

5. High-Level Requirements

Requirement

Ingest market data, fundamental data, filings/news, and derived technical indicators o scheduled basis.

Create candidate universes using liquidity, price, and market-cap filters.

Generate event-aware and technical signals, then combine them into a normalized sco

Compute deterministic entry zone, stop-loss, take-profit, and trade invalidation condit

Reject ideas that fail risk, liquidity, concentration, or policy checks.

Publish daily recommendation objects through API and report formats.

1 All recommendation decisions must be reproducible from point-in-time data snapshot

2 The platform must support backtesting, walk-forward testing, and paper-trading before production execution.

3 System components must be modular enough to replace data vendors, models, or orchestration frameworks without redesigning the full platform.

6. System Context and Reference Architecture

The architecture separates information ingestion, signal generation, price-plan construction, recommendation synthesis, and policy guardrails. This avoids the common failure mode where a general-purpose LLM directly issues trades without deterministic validation.

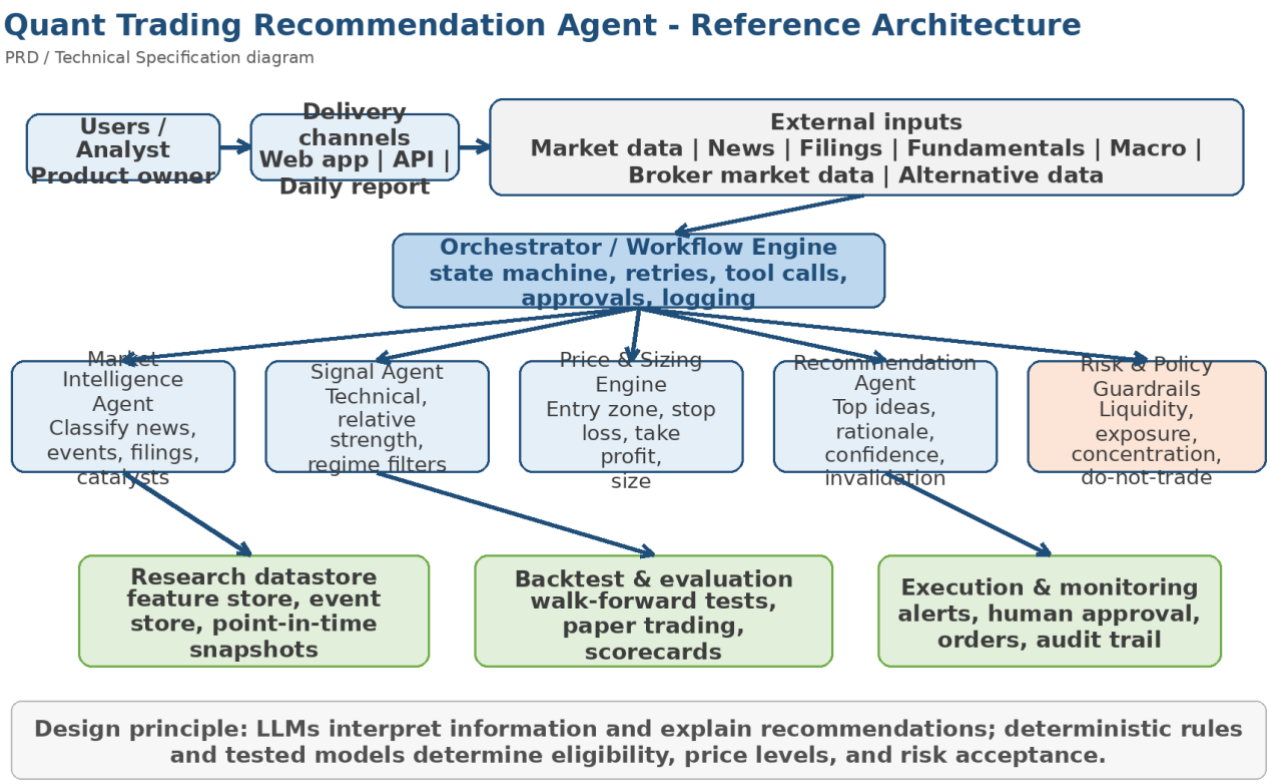

Figure 1. Reference architecture for the quant research and trading recommendation agent.

7. Agent Responsibilities

Component	Responsibility	Inputs	Outputs
Market Intelligence Agent	Parse news, earnings, filings, macro events, and company-specific catalysts into structured event objects with sentiment, relevance, and time horizon.	Raw news/feed items, filing text, company metadata	Event objects, catalyst summaries, entity-level impact scores
Signal Agent	Compute technical, cross-sectional, and regime-aware signals from price/volume and derived features.	OHLCV, sector benchmarks, index data, event objects	Normalized signal vectors and composite signal score
Price & Sizing Engine	Turn signal outputs into executable	Signal score, ATR,	Trade plan object with

	trade plans using deterministic logic for entry, stop, target, and position size.	support/resistance, liquidity metrics	entry zone, stop, targets, risk/reward
Recommendation Agent	Explain why a name is attractive, summarize thesis, confidence, and invalidation conditions.	Trade plan object and supporting evidence	Human-readable recommendation package, API payload, daily report
Risk & Policy Guardrails	Apply exposure, liquidity, event blackout, concentration, and confidence thresholds.	Trade plan, portfolio state, compliance rules	Approved or rejected recommendation with reason codes

8. Core Recommendation Object

All downstream consumers receive the same typed recommendation schema so the system can support reporting, review, backtesting, and execution without format drift.

```
{ "ticker": "AAPL", "direction": "BUY", "entry_zone": [186.5, 188.2],
  "stop_loss": 182.9, "take_profit_zone": [194.0, 198.0],
  "holding_period": "3-10 trading days", "confidence": 0.73, "risk_level": "medium",
  "thesis": ["estimate revisions positive", "breakout confirmed", "sector leadership
improving"],
  "invalid_if": ["close below 20D low", "broad market regime turns risk-off"],
  "source_snapshot_id": "2026-04-10T09:30:00Z#batch001" }
```

9. Functional Flow

1. Load point-in-time data snapshots for market data, fundamentals, and event feeds.
2. Build the tradable universe using liquidity, price, spread, and market-cap rules.
3. Parse new information into event objects with entity tags and catalyst labels.
4. Compute technical and cross-sectional features, then evaluate regime filters.
5. Rank securities using a weighted score or trained ranking model.
6. Generate deterministic entry zone, stop-loss, and sell target levels.
7. Apply risk and policy guardrails. Reject any trade plan that fails.
8. Publish approved recommendations to API, dashboard, and daily report.
9. Record full evidence and scores for later attribution and backtesting.

10. Signal Engine Design

The signal engine should treat news and price action as separate evidence streams. A practical composite score for the first release is a weighted linear model, with enough transparency to support debugging and post-trade review.

Composite Score = 0.30 Technical + 0.25 Event/News + 0.20 Relative Strength + 0.15 Fundamental + 0.10 Execution Quality

Candidate sub-signals may include trend breakout strength, pullback quality, volatility compression, earnings revision trend, sector-relative momentum, benchmark-relative performance, and event freshness.

11. Price-Level Generation Rules

Pattern	Entry logic	Stop logic	Sell logic
Breakout	Previous high breakout ± 0.3 ATR or first orderly retest of breakout level	Below local swing low or 1.2-1.8 ATR	2R first target, then prior resistance or ATR trail
Pullback	Retest of 10/20 EMA, anchored VWAP, or support zone with improving tape	Below support zone or failed retest low	Return to prior high, then trend-following trail
Mean reversion	Only in neutral/risk-on regimes; discount to lower volatility band with reversal confirmation	Below lower structure break	Middle band, prior mean, or fixed reward multiple
Short setup	Breakdown or failed bounce beneath resistance in weak relative-strength regime	Above failed bounce high or 1.2-1.8 ATR	2R target, support zone, then trail

12. Risk and Policy Guardrails

- No recommendation for securities below minimum liquidity, minimum price, or maximum spread thresholds.
- No recommendation during defined blackout windows, such as minutes before earnings or material unscheduled news, unless strategy explicitly supports event trading.
- Portfolio-level caps on name concentration, sector concentration, gross exposure, and correlated risk.
- Hard rejection when recommendation confidence is below threshold or when evidence sources materially disagree.
- Human approval required before any live order routing in early production phases.

13. Data Model and Storage

The platform should preserve point-in-time data fidelity. Every recommendation must be linked to the exact market snapshot, event payloads, feature values, and model version used to produce it.

Store	Purpose	Key entities

Market snapshot store	Immutable point-in-time prices and vendor-normalized fields	ticker, timestamp, OHLCV, corporate actions, vendor id
Event store	Parsed news, filings, catalysts, and sentiment labels	event id, entity tags, event type, sentiment, horizon, source url
Feature store	Precomputed technical, cross-sectional, and macro features	feature name, lookback, value, training/serving parity
Recommendation store	Published recommendation objects and evidence	trade id, score vector, entry/stop/target, confidence
Audit / execution store	Approvals, alerts, paper trades, and live orders	decision id, approver, route, slippage, P&L attribution

14. API Surface

Endpoint	Purpose
GET /universe	Returns the current tradable universe after filters.
POST /research/run	Triggers a research batch for a specified market date or intraday window.
GET /recommendations	Returns approved recommendation objects with filters for date, ticker, score, and status.
GET /recommendations/{id}	Returns full supporting evidence, feature values, and model metadata.
POST /paper-orders	Simulates trade execution using approved recommendation objects.
GET /metrics	Returns backtest, paper-trading, and operational health metrics.

15. Evaluation Framework

- Backtest with point-in-time inputs and realistic transaction costs.
- Walk-forward validation rather than single split optimization.
- Paper-trading stage with recommendation publication before live deployment.
- Post-trade attribution: signal quality, event contribution, execution slippage, and stop/target hit paths.
- Operational metrics: runtime, missing data rate, recommendation rejection rate, and explanation latency.

16. Success Metrics

Category	Metrics
Research quality	Hit rate, average return per recommendation, win/loss ratio, expectancy, maximum drawdown
Decision quality	Calibration of confidence scores, rejection precision, thesis invalidation accuracy
Execution quality	Paper-trade slippage, fill simulation quality, timeliness of recommendation publication
Platform quality	Batch success rate, data freshness SLA, feature parity failures, model drift alerts

17. MVP Scope

- US equities only, daily bars plus optional delayed intraday data.
- Single-direction recommendations (BUY first; SHORT optional later).
- Deterministic ranking model plus event parser and basic LLM explanations.
- No unattended live execution. Support only dashboard, API, and paper-trading outputs.
- Human review screen for final approval of published ideas.

18. Implementation Roadmap

Phase	Deliverable
Phase 0 - Design	Finalize universe rules, score decomposition, and recommendation schema.
Phase 1 - Data foundation	Integrate historical prices, vendor normalization, event feed storage, and feature pipelines.
Phase 2 - Research MVP	Build signal engine, price-plan rules, recommendation API, and daily report.
Phase 3 - Validation	Run walk-forward tests, paper trading, and model/feature monitoring.
Phase 4 - Controlled execution	Add broker integration behind approval gates, throttles, and rollback controls.

19. Reference Open-Source Inspirations

The target design intentionally borrows patterns from established open-source repositories rather than reinventing every layer. The selected references below informed the modular decomposition of data integration, backtesting, agent orchestration, and research automation.

Project	GitHub stars*	Use as reference for	Notes
OpenBB	65.6k	data platform / research integration	Open-source data toolset for analysts, quants, and AI agents; useful for the ingestion and data-service layer.
Backtrader	21.1k	backtesting engine	Mature Python backtesting framework suitable for research iteration and walk-forward validation.
Freqtrade	49k	execution and strategy workflow	Useful reference for bot operations, strategy packaging, hyperopt, and monitoring patterns, even though it is

			crypto-first.
LangChain	133k	LLM component framework	Provides reusable abstractions for tool calling and agentic workflows; useful if a flexible orchestration stack is preferred.
CrewAI	48.5k	multi-agent orchestration	Useful reference for role-based agent decomposition and workflow composition.
FinRL	14.7k	RL/quant research	Useful for financial reinforcement learning experiments and for understanding train-test-trade separation.

* Stars observed on GitHub pages accessed on 10 April 2026; values change over time.

20. Risks and Open Questions

- Which market data and news vendors are legally and economically viable for production use?
- Should ranking remain interpretable and mostly deterministic, or should later phases introduce learned ranking models?
- What approval workflow is needed before moving from paper trading to controlled live routing?
- How should the system separate overnight swing trades from intraday opportunities and event-driven setups?
- What minimum confidence threshold is acceptable before publishing any recommendation to end users?

Appendix A. Source References

- OpenBB-finance/OpenBB. GitHub repository. Accessed 10 April 2026.
- mementum/backtrader. GitHub repository. Accessed 10 April 2026.
- freqtrade/freqtrade. GitHub repository. Accessed 10 April 2026.
- langchain-ai/langchain. GitHub repository. Accessed 10 April 2026.
- crewAIInc/crewAI. GitHub repository. Accessed 10 April 2026.
- AI4Finance-Foundation/FinRL. GitHub repository. Accessed 10 April 2026.

21. Implementation Plan

Purpose. This section translates the product requirements and technical architecture into an execution plan that an engineering team can implement in staged releases. The plan assumes a research-first deployment model: recommendation generation, backtesting, paper trading, and only then controlled live routing.

21.1 Delivery Principles

- Ship in thin vertical slices. Each milestone should produce a working system, not only framework code.
- Keep LLM responsibilities narrow. Language models summarize, classify, and explain; deterministic code computes prices, signals, risk limits, and routing decisions.
- Make every recommendation reproducible. Each output must be traceable to source data versions, feature values, model versions, and prompt versions.
- Design for auditability from day one. Recommendation records, order decisions, and overrides should be persisted as immutable events.
- Productionize only after evidence. New strategies or ranking models move from research to paper trading only after backtest, walk-forward, and operational validation.

21.2 Proposed Repository Structure

```
quant-agent/
  apps/
    api/            # FastAPI service
    worker/         # scheduled jobs and async tasks
    dashboard/      # internal review UI
  services/
    ingestion/      # market, fundamentals, news, filings
    features/       # indicators, event features, regime features
    ranking/        # candidate ranking and recommendation logic
    risk/           # risk filters and portfolio constraints
    execution/      # paper routing and broker adapters
    research/       # backtests, walk-forward, experiments
    llm/            # prompts, schemas, evaluators, guardrails
  domain/
    entities/       # Recommendation, Signal, Position, Event
    policies/       # business rules and approval policies
  infra/
    db/             # migrations and persistence models
    cache/          # Redis integration
    observability/  # logging, metrics, tracing
  tests/
    unit/
    integration/
    regression/
  docs/
    prd/
    runbooks/
```

prompts/

21.3 Phase-by-Phase Roadmap

Phase 0: Foundations (1-2 weeks)

- Initialize the monorepo, CI pipeline, code quality rules, secrets management, and environment templates.
- Provision PostgreSQL, Redis, object storage, and a metrics stack for local and staging use.
- Define core schemas for MarketBar, NewsEvent, SignalSnapshot, Recommendation, PaperOrder, Fill, and PositionState.
- Implement provider abstraction interfaces so data vendors and brokers can be swapped without changing business logic.

Phase 1: Data and Research MVP (2-4 weeks)

- Ingest daily and intraday OHLCV, corporate actions, benchmark data, and sector classifications.
- Add a first news pipeline for headline retrieval, ticker mapping, deduplication, and event timestamp normalization.
- Build feature jobs for trend, momentum, volatility, liquidity, and relative strength metrics.
- Create a research notebook and batch backtest harness to score candidate ranking rules on historical windows.
- Deliver an internal report that lists the top ten trade ideas with entry zone, stop, take-profit, and rationale.

Phase 2: Recommendation Engine (2-3 weeks)

- Implement deterministic entry, stop-loss, and take-profit generators using ATR bands, support/resistance, and regime-specific templates.
- Introduce the Recommendation Builder service that merges signal scores, event scores, and risk filters into one ranked decision object.
- Add LLM-assisted event summarization and explanation generation behind strict JSON schemas and automated validation.
- Persist recommendation versions and render them through an internal dashboard and API.

Phase 3: Paper Trading and Monitoring (2-4 weeks)

- Create a paper execution router with order simulation, slippage assumptions, fill logic, and position bookkeeping.
- Add recommendation expiry, trading session windows, and cancel/replace behavior for stale ideas.
- Build daily monitoring jobs for hit rate, average excursion, stop-loss behavior, and PnL by signal family.
- Introduce approval gates before a recommendation can be promoted from research output to paper routing.

Phase 4: Controlled Live Routing (optional, after evidence)

- Integrate one broker only after paper-trading stability, logging completeness, and human approval workflows are in place.
- Start with low-notional routing, single-market coverage, and restricted trading windows.
- Keep all live orders behind kill switches, broker health checks, and explicit human override capability.

21.4 Service Build Order

Order	Service	Why it comes now
1	Market data ingestion	Normalize bars, symbols, calendars, splits, and dividends.
2	Feature engine	Compute reusable features with point-in-time correctness.
3	Event intelligence	Map news and filings to tickers, sentiment, and event types.
4	Signal scoring	Generate candidate scores and rank eligible names.
5	Risk engine	Reject names violating liquidity, exposure, or event-risk rules.
6	Recommendation builder	Produce entry/stop/target zones and explanation payloads.
7	Backtest engine	Evaluate edge, drawdown, turnover, and walk-forward stability.
8	Paper execution	Track paper orders, fills, and realized recommendation outcomes.
9	Dashboard and API	Expose outputs for review, approval, and operational monitoring.

21.5 Core User Stories

- As a researcher, I can rerun a historical day and reproduce the exact recommendation list that the system would have generated at that time.
- As a reviewer, I can inspect each recommendation with its raw inputs, computed features, explanation text, and risk flags before approval.
- As an operator, I can see which recommendations expired, were paper-traded, hit stop-loss, or reached target, with full timestamps.
- As a developer, I can add a new signal factor or event classifier without changing unrelated services.

21.6 API and Job Plan

API endpoints: /recommendations/latest, /recommendations/{id}, /signals/{ticker}, /backtests/runs, /paper-orders, /positions, /health

Scheduled jobs: pre-market ingestion, intraday refresh, end-of-day reconciliation, nightly backtest batch, daily metrics aggregation

Message queues: feature recomputation events, recommendation-ready events, paper fill events, model-evaluation events

21.7 Data Model Priorities

Recommendation: id, generated_at, ticker, direction, entry_zone_low, entry_zone_high, stop_loss, tp1, tp2, confidence, risk_grade, explanation, feature_snapshot_id

Signal Snapshot: ticker, timestamp, trend_score, momentum_score, volatility_score, liquidity_score, relative_strength_score, event_score, regime_label

Event Record: source_id, published_at, ingested_at, headline, normalized_text, tickers, event_type, sentiment, relevance, llm_summary_version

Paper Order: recommendation_id, side, qty, limit_price, submitted_at, status, simulated_fill_price, filled_at, cancel_reason

Position State: ticker, open_time, avg_price, qty, realized_pnl, unrealized_pnl, stop_state, target_state, last_mark

21.8 Engineering Tasks by Module

Ingestion

- Implement exchange/calendar normalization and symbol master management.
- Backfill historical bars and validate point-in-time corporate actions handling.
- Add data quality checks for stale bars, zero volume anomalies, and missing sessions.

Features

- Package indicators into reusable transforms with deterministic inputs.
- Version features so model and ranking logic can be tied to a specific feature set.
- Cache expensive feature windows without losing reproducibility.

LLM/Event Intelligence

- Define structured prompts and JSON schema validation.
- Create offline evaluation sets for event labeling accuracy.
- Add fallback deterministic rules when model output is invalid or unavailable.

Risk

- Implement hard limits for liquidity, spread, earnings blackout windows, sector exposure, and max open positions.
- Create a separate approval policy object so thresholds can change without code rewrites.

Research

- Build a backtest CLI and parameterized experiment runner.
- Log evaluation metrics, configuration hashes, and artifact links for each run.

Execution

- Simulate market and limit order behavior under configurable slippage assumptions.
- Add idempotent order processing and reconciliation between recommendation state and position state.

21.9 Test Strategy

- Unit tests for feature calculations, pricing zone generation, and risk-policy evaluation.
- Contract tests for provider adapters and broker/paper router interfaces.
- Regression tests that replay fixed market days and compare recommendation outputs against approved snapshots.
- Backtest sanity tests to detect look-ahead bias, survivorship leakage, and timestamp misalignment.
- LLM evaluation tests for schema compliance, label consistency, and fallback-path behavior.

21.10 Definition of Done by Release

MVP

- Daily recommendation list generated automatically for a defined universe.
- Each recommendation includes entry, stop, target, and explanation.
- Research backtest report available for at least one year of historical data.

Paper Trading

- Recommendations flow into simulated orders and tracked outcomes.
- Operational dashboard shows health, latency, and realized paper performance.
- Manual approval and kill-switch flows are tested.

Live Pilot

- Broker adapter passes integration testing.
- All actions are logged and reversible where operationally possible.
- Pilot remains within approved notional and exposure limits.

21.11 Suggested OpenCode / Codex Execution Prompts

Create the FastAPI service skeleton, SQLAlchemy models, Alembic migrations, and health endpoints for the quant-agent repository structure defined in the PRD.

Implement a market data ingestion module with provider interfaces, daily OHLCV normalization, symbol master support, and data quality checks.

Build a feature engine that computes ATR, moving averages, relative strength, volatility, and liquidity features with deterministic tests.

Implement a Recommendation Builder that converts signal scores into entry zones, stop-loss levels, and take-profit targets using configurable rules.

Create a paper trading router with simulated fills, position tracking, and idempotent order handling.

Add regression tests that replay a historical market date and verify recommendation outputs against a stored snapshot.

21.12 Immediate Next Actions

- Decide the initial market scope: U.S. equities only, or equities plus ETFs.
- Choose the first data provider combination for prices, news, and fundamentals.
- Lock the first recommendation horizon, such as 1-5 day swing trades.
- Approve the MVP output schema so engineering can code against a stable contract.
- Start with paper trading by default and defer broker integration until the review committee signs off.